Communcation systems project

Negin Raki , 402101758 Department of Electrical Engineering, University of Sharif ,

2.System's Structure

0.1 Information Source

Part 1: Theoretical Entropy Rate Calculation

Consider a discrete information source with the alphabet and symbol probabilities shown in Table 1.

Symbol	Probability
A	0.25
B	0.20
C	0.15
D	0.10
E	0.08
F	0.07
G	0.05
H	0.04
I	0.04
J	0.02

Table 1: Source alphabet and symbol probabilities

The entropy of the source is given by Shannon's formula:

$$H(X) = -\sum_{i} p_i \log_2(p_i) \tag{1}$$

Substituting the given probabilities, the entropy rate becomes: H(X) = -($0.25 \log_2 0.25 + 0.20 \log_2 0.20 + 0.15 \log_2 0.15 + 0.10 \log_2 0.10$

$+ 0.08 \log_2 0.08 + 0.07 \log_2 0.07 + 0.05 \log_2 0.05 + 0.04 \log_2 0.04$

$+ 0.04 \log_2 0.04 + 0.02 \log_2 0.02) = 2.97 bit/symbol$

which represents the theoretical minimum average bit rate required to encode the source.

Part 2: Entropy Rate function:

The result of written function is as the same as our calculation in previous part.

0.2 Coding Source

Huffman Coding for the Source

Optimized Huffman Coding

Although Huffman coding minimizes the average code length, different optimal Huffman trees may exist depending on how ties are resolved. By constructing a more balanced Huffman tree, the maximum codeword length can be reduced while preserving optimality.

A balanced Huffman code for the source is shown in Table 2.

Symbol	Probability	Huffman Code
A	0.25	11
B	0.20	10
C	0.15	011
D	0.10	010
E	0.08	0011
F	0.07	0010
G	0.05	00011
H	0.04	00010
I	0.04	00001
J	0.02	00000

Table 2: Balanced Huffman codes with reduced maximum length

The resulting code is prefix-free and achieves the minimum possible average code length while reducing the maximum codeword length to five bits, making it more practical for implementation.

Explanation: Huffman coding assigns shorter codes to symbols with higher probability and longer codes to symbols with lower probability, minimizing the average code length. The resulting code is prefix-free, meaning no code is a prefix of another code.

The average code length can be calculated as:

$$L_{avg} = \sum_{i} p_i \, l_i \tag{2}$$

where l_i is the length of the Huffman code for symbol i. This value is very close to the source entropy, achieving near-optimal compression.By using the designed huffman codefor these symboles we will get the average length 3 bit per symbol which is quite close to the entropy and also its more efficent than previous result.

0.3 Codes' Explanation:

first we should write code for finding the most efficient bit symboizaition using Huffman tree , for this we use *heapq* library for finding the least probability in huffman trees.

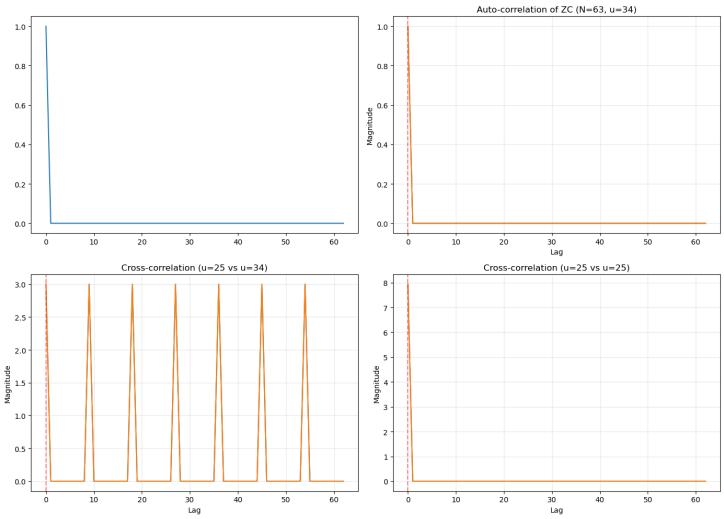


Figure 1: Result of using huffman trees

0.4 Channel Coding

In this work, a **Hamming**$(7, 4)$ linear block code is employed for channel coding in order to detect and correct single-bit errors caused by channel noise. This code maps 4 information bits into a 7-bit codeword by adding 3 parity bits.

Codeword Structure

The positions of data and parity bits in the Hamming$(7, 4)$ codeword are shown in Table 3.

Bit position	1	2	3	4	5	6	7
Bit type	P_1	P_2	D_1	P_3	D_2	D_3	D_4

Table 3: Bit positions in the Hamming$(7, 4)$ codeword

Parity bits are placed at positions that are powers of two (positions 1, 2, and 4), while the remaining positions carry the data bits.

Parity Check Equations

Each parity bit is computed to ensure even parity over a specific subset of codeword bits:

- P_1 checks bits: $1, 3, 5, 7$
- P_2 checks bits: $2, 3, 6, 7$
- P_3 checks bits: $4, 5, 6, 7$

Syndrome Calculation and Error Correction

At the receiver, the error syndrome vector is calculated using the parity-check equations:

$$\mathbf{S} = (s_3\ s_2\ s_1)_2$$

where s_1, s_2, and s_3 represent the results of the parity checks corresponding to P_1, P_2, and P_3, respectively.

The syndrome $\mathbf{S}$ forms a binary number whose decimal value indicates the position of a single-bit error in the received codeword:

- $\mathbf{S} = 000$ indicates that no error has occurred,
- $\mathbf{S} \neq 000$ identifies the index of the erroneous bit.

Once the erroneous bit position is determined, error correction is performed by flipping the corresponding bit. The corrected codeword is then decoded to recover the original 4 information bits.

This capability makes the Hamming$(7, 4)$ code a **single-error correcting (SEC)** code, which significantly improves reliability in noisy communication channels.

THE EXPLANATIONS OF FROM PART 2.7 TO 4 ARE IN NOTEBOOK.

PART4:PCM STIMULATION WITH COMPANDING

0.5 Sound signals sampling:

By using **sounddevice** library in python we can record our voice in order to analyze it in time and frequency domain.

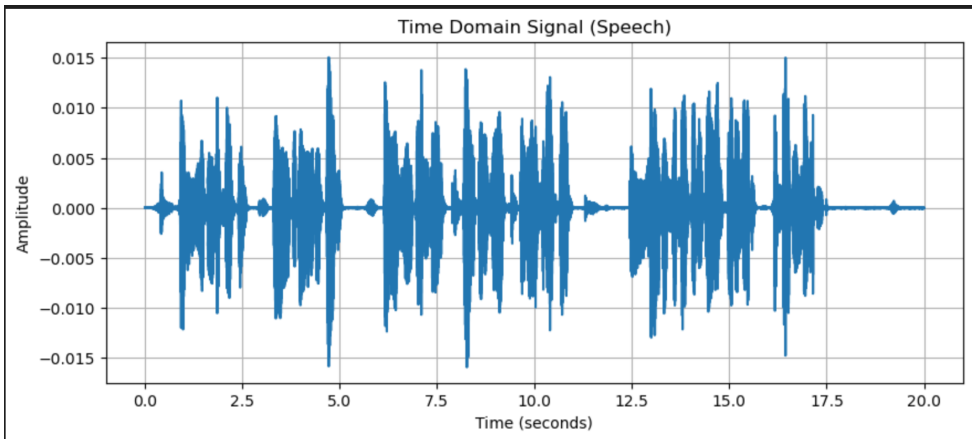


Figure 2: The signal of voice in time domain

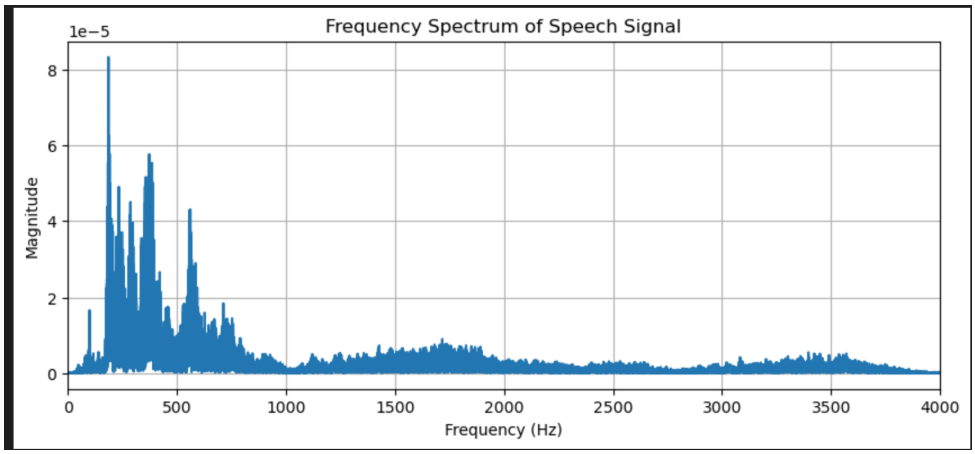


Figure 3: The signal of voice in frequency domain

”As we can see while the human voice contains frequencies up to several kilohertz, standard PCM telephony limits the analog bandwidth to 4 kHz. Following the Nyquist theorem, the signal is sampled at 8 kHz. When quantized at 8 bits per sample (2^8 levels), this results in a digital bit rate of 64 kbps for the transmitted signal.”

0.6 PCM QUANTIZATION

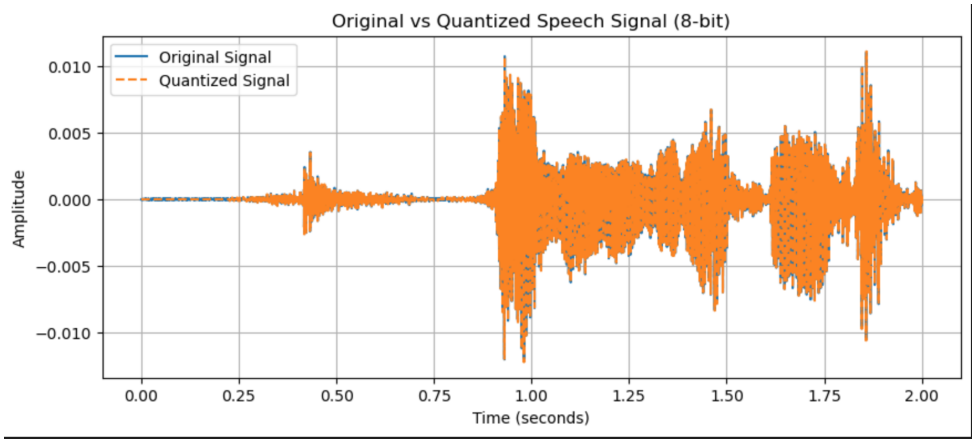


Figure 4: Qunatized siganl and original one

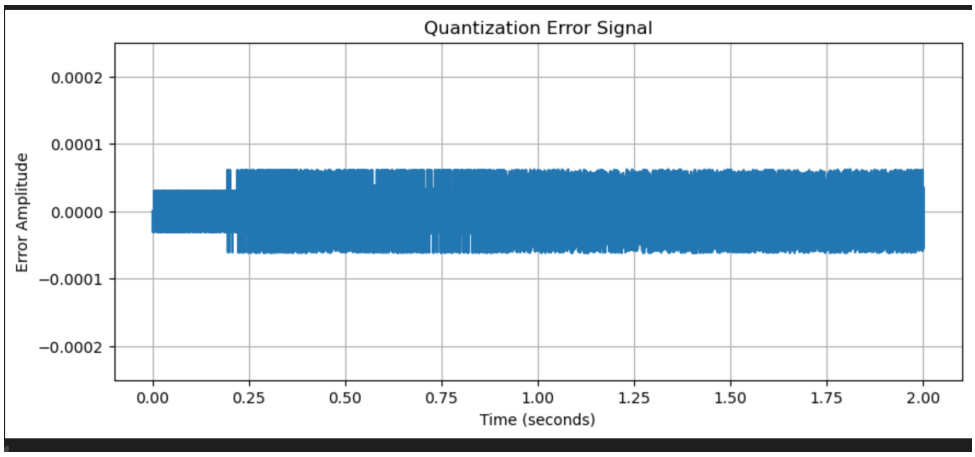


Figure 5: Quantization Error signal: AS we know Quantization error stays constant,no matter how high the magnitude of original signal becomes.

```python
def UniformQuantizer(x, n_bits):

    L = 2 ** n_bits                     # number of levels
    x_max = np.max(np.abs(x))           # dynamic range
    delta = 2 * x_max / L               # step size

    xq = delta * np.round(x / delta)    # quantization
    return xq
```

✓ 0.0s

Figure 6: In uniformQuantizer function we use delta for normalization and also we round every x /delta in time vector to find the closest level to the actual signal magnitude at that time.

```python
def SNR(original, reconstructed):
    noise = original - reconstructed
    snr = 10 * np.log10(np.sum(original**2) / np.sum(noise**2))
    return snr
```

✓ 0.0s

Figure 7: SNR function : Energy ratio is used to find the SNR(dB)

```python
def Compressor(x, mu=255):
    x_max = np.max(np.abs(x))
    x_norm = x / x_max
    return np.sign(x_norm) * (np.log(1 + mu * np.abs(x_norm)) / np.log(1 + mu))

def Expander(y, mu=255):
    return np.sign(y) * (1/mu) * ((1 + mu)**(np.abs(y)) - 1)
```
✓ 0.0s

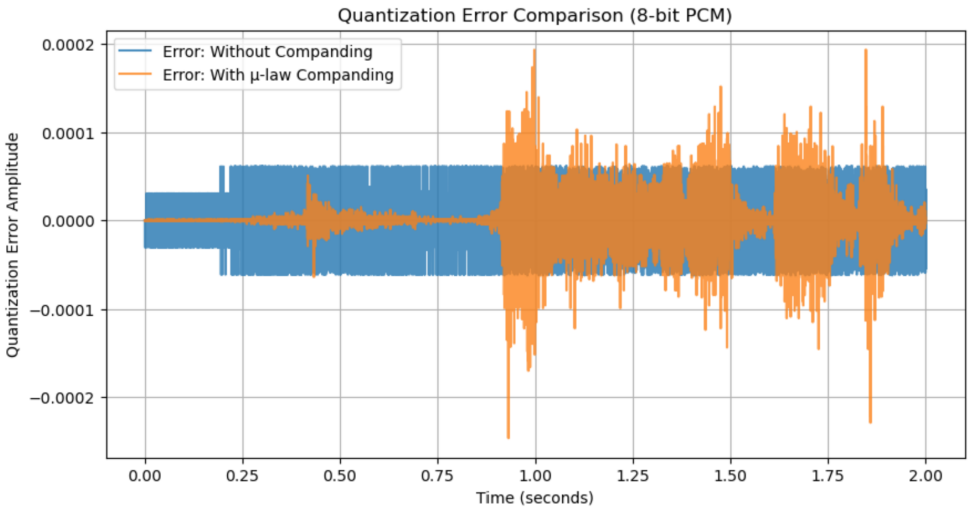
Figure 8: by using the compressor first we normalize the signal the pass it through the formula of U$_{l}awcompandingsignalafterthatweexpandsignalagain$.

0.7 PCM QUANTIZATION WITH COMPANDING:

Now, if we get SNR of companded signal we can see SNR rises from 35.66db to 38 db. which gives us sound signal with better quality.

0.8 Results:

Q1. In Uniform Quantization, the error is spread evenly across all signal levels. Whether



the person is whispering or shouting, the ”step size” of the error is the same. In Companded Quantization, the error is much smaller for quiet sounds (low amplitudes) and larger for loud sounds (high amplitudes). When you plot this, you’ll see that Companding ”tightens” the error distribution around the center (zero) where most speech happens.

Q2.Small signals suffer the most in standard PCM because the quantization steps are too big compared to the signal itself, leading to quantization noise. Companding solves this by using very fine (small) steps for low-amplitude signals. This significantly reduces the noise for quiet parts of the audio, making the background noise much less noticeable

Q3.Uniform PCM: Sounds okay for loud voices, but quiet voices sound "robotic" or distorted because of the high relative noise. Companded PCM: Sounds much more natural and consistent. It maintains a high Signal-to-Quantization-Noise Ratio (SQNR) across a wide range of volumes. To the human ear, the Companded signal sounds "cleaner" overall.

Q4.Efficiency: Companding allows us to get high-quality sound using only 8 bits per sample. Companding mimics how we actually hear, putting the "detail" where our ears notice it most. It's the industry standard (using μ-law or A-law) because it makes long-distance phone calls sound clear without needing massive amounts of data.

PART5:RACH stimulation using ZADOFFCHU

0.9 AUtocorrelation and correlation:

Similarities:

Constant Amplitude: Both auto and cross-correlation involve sequences with a constant magnitude ($|x[n]| = 1$), which ensures a flat frequency spectrum.

Zero-Lag Peak: When comparing a sequence to itself (or an identical copy), both will result in a maximum magnitude peak at zero lag.

Differences: Sidelobe Levels: Ideal autocorrelation has zero sidelobes at all non-zero lags. Cross-correlation between different roots u_1 and u_2 results in a non-zero, constant floor of magnitude $1/\sqrt{N}$ (if N is prime).

Sensitivity to Shift: Autocorrelation is used to find the exact timing (arrival time) because of its unique peak. Cross-correlation is used to distinguish between different users or cells.

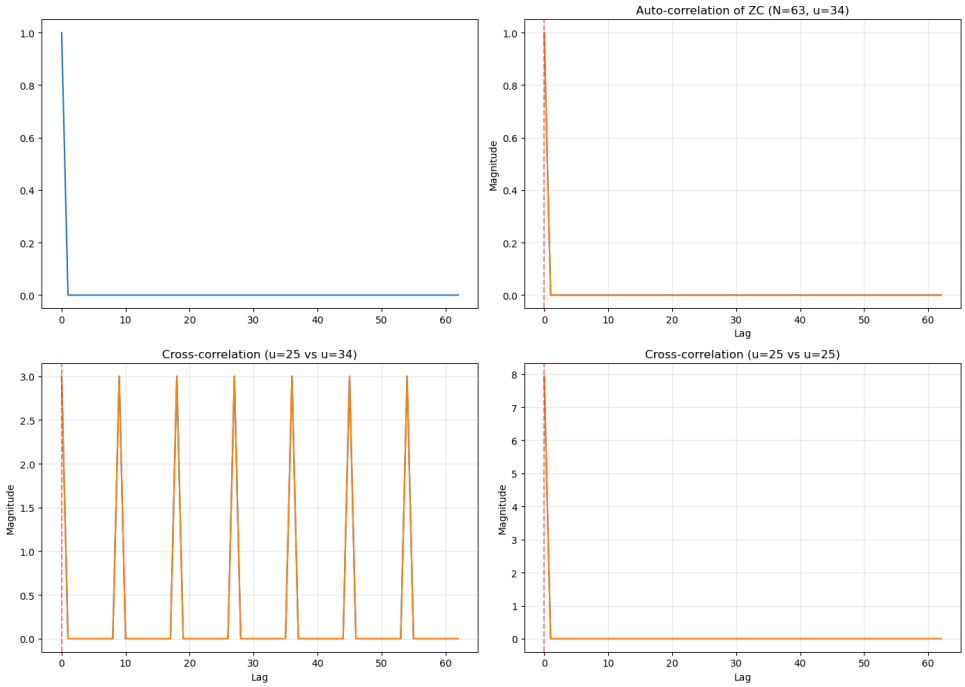


Figure 9: correlatinos

Q4. Low Interference (Cross-correlation): In a cell, different users are assigned sequences with different roots. Because the cross-correlation between different roots is very low, the Base Station can distinguish between multiple users trying to connect at the same time without them ”jamming” each other.

Timing Estimation (Autocorrelation): The ”impulse-like” autocorrelation allows the Base Station to determine the exact arrival time of a signal. This is crucial for calculating the Timing Advance (TA) needed to synchronize the user’s upload.

Low PAPR (Peak-to-Average Power Ratio): Since the sequence has a constant envelope, it doesn’t strain the battery of the mobile phone and allows the transmitter to operate efficiently at high power.

0.10 MIB detection and Random access procedure:

1 Phase 1: MIB Detection (Synchronization)

The MIB is critical for the UE to acquire initial system parameters. In this simulation:

- **Data Generation:** A 1000-bit stream was generated, serving as the payload.
- **Pattern Insertion:** A unique 24-bit MIB pattern was inserted at index 287 to simulate the Physical Broadcast Channel (PBCH) structure.
- **Modulation:** QPSK modulation was applied to map bits to complex symbols $s \in \{\pm 1 \pm j\}/\sqrt{2}$.
- **Detection:** A sliding window correlation was used at the receiver. The correlation peak identifies the exact timing of the frame.

```
# MIB Detection using correlation
def detect_pattern(rx_bits, pattern):
    """Detect pattern in received bits using correlation"""
    # Convert to float for correlation
    rx_float = 2 * rx_bits.astype(float) - 1  # Convert to ±1
    pattern_float = 2 * pattern.astype(float) - 1

    # Compute correlation
    corr = np.correlate(rx_float, pattern_float, mode='valid')
    corr = corr / len(pattern)  # Normalize

    # Find peak
    peak_pos = np.argmax(np.abs(corr))
    peak_value = corr[peak_pos]

    return peak_pos, peak_value, corr

# Detect MIB
detected_pos, corr_value, all_corr = detect_pattern(rx_bits, mib_patterr
```

Figure 10: detection function

2 Phase 2: RACH Preamble Generation and Detection

The RACH process uses Zadoff-Chu (ZC) sequences due to their Constant Amplitude Zero Auto-Correlation (CAZAC) properties.

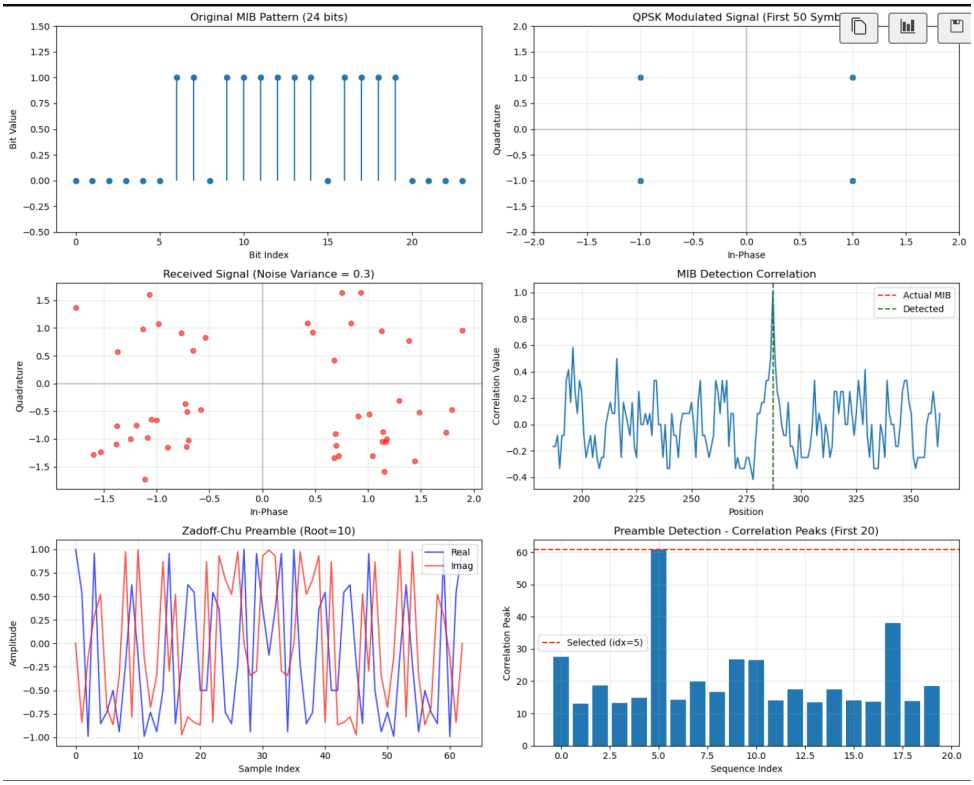
2.1 Simulation Parameters

- **Sequence Length (N):** 63.
- **Root Selection:** Root $u = 10$ was assigned to Preamble Index 5.
- **Detection Mechanism:** The receiver computes the circular cross-correlation between the received signal and all possible root sequences in the cell's codebook.

```
# MIB Detection using correlation
def detect_pattern(rx_bits, pattern):
    """Detect pattern in received bits using correlation"""
    # Convert to float for correlation
    rx_float = 2 * rx_bits.astype(float) - 1  # Convert to ±1
    pattern_float = 2 * pattern.astype(float) - 1

    # Compute correlation
    corr = np.correlate(rx_float, pattern_float, mode='valid')
    corr = corr / len(pattern)  # Normalize

    # Find peak
    peak_pos = np.argmax(np.abs(corr))
    peak_value = corr[peak_pos]

    return peak_pos, peak_value, corr

# Detect MIB
detected_pos, corr_value, all_corr = detect_pattern(rx_bits, mib_patterr
```

3 Conclusion

The UE successfully established a connection by detecting the MIB and completing the 4-step RACH handshake simulation. The use of Zadoff-Chu sequences ensured low interference and



accurate timing estimation even in the presence of AWGN noise.